Static Analysis for Bounds Check Elision in the Naga WGSL Compiler

Bhuvan B

bhuvanb1408@gmail.com

Abstract. The WebGPU standard and its shading language WGSL mandate compile-time bounds checking for all GPU memory accesses, a design choice that distinguishes them from traditional GPU programming models (CUDA, OpenCL, GLSL). The Naga compiler, used in wgpu and Mozilla Firefox, provides three conservative bounds check policies (ReadZeroSkipWrite, Restrict, Unchecked) with different safety and performance characteristics.

This work presents a static analysis pass that elides unnecessary bounds checks by exploiting specification-level guarantees and common loop patterns. The builtin index safety analysis verifies that accesses indexed by local_invocation_index are provably safe when the array length meets or exceeds the total workgroup size, leveraging a guarantee inherent to the WGSL specification. The loop induction variable analysis recognizes if $i < N$ { } else { break; } patterns inside WGSL loops to identify further safe accesses. Both analyses produce SPIR-V output byte-identical to the Unchecked policy for elided cases, achieving zero runtime overhead while preserving full safety for all other accesses.

1 Introduction

With web browsers and compute on web becoming increasingly popular , graphics and compute shaders are deployed through web applications. From WebGL to current WebGPU standards, scientific, financial computations, mechanical simulations and medical applications are being deployed to be compatible with web and mobile GPU compute. The WebGPU standard and its shading language WGSL were designed with safety as a first-class concern, in contrast to other GPU programming models (CUDA, OpenCL, legacy GLSL) that treat out-of-bounds (OOB) memory access as undefined behavior [1, 2].

The need for availability of resources irrespective of results from a shader result mandates that OOB failures that break and crash live memory are priority concerns. The Naga shader compiler [3] translates WGSL to SPIR-V and provides configurable bounds check policies that insert runtime guards on every array and buffer access. While these policies prevent OOB memory corruption, they introduce instruction overhead that may be eliminated when the compiler can statically prove an index is within bounds.

This paper and ongoing research envisions to solve this thorough optimizations that do not give correctness penalty. Correct work includes:

- Two static analyses for bounds check elision in the Naga compiler: a builtin index safety analysis that exploits the WGSL specification's guarantee on local_invocation_index, and a loop induction variable analysis targeting WGSL loop patterns.
- Currently targeting vulkan. Future work will include expanding to other backends.

2 Background About Findings

2.1 Bounds Check Policies in Naga

Naga defines three BoundsCheckPolicy variants [4]:

ReadZeroSkipWrite Injected bounds checks return 0 for OOB reads and silently discard OOB writes.

This is the safest and most expensive policy.

Restrict Indices are clamped to $[0, \text{length} - 1]$ before access. OOB reads return the last element; OOB writes overwrite the last element.

Unchecked No bounds checks are inserted. OOB access produces undefined behavior identical to CUDA and raw Vulkan. Used in native workloads, not opted in browser-specific applications.

Figure 1 shows where these policies are applied in the compilation pipeline.

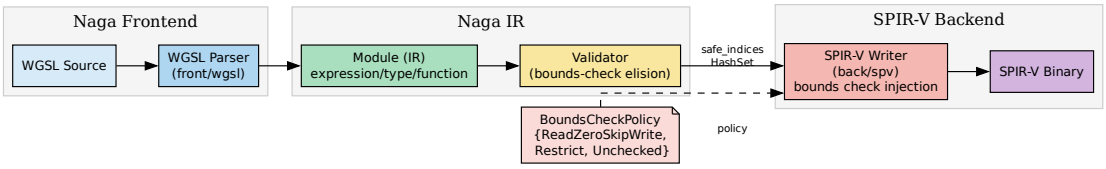


Figure 1: Naga compilation pipeline with bounds check injection.

2.2 SPIR-V Instruction Model

SPIR-V generation: A WGSL shader once compiled generated SPIR-V code that runs on Vulkan for a linux system. The bounds check happens for every array access at any index i on a length N var<workgroup> as illustrated below:

```
; Bounds check: i < N?
%cmp = OpULessThan %bool %i %N
     OpSelectionMerge %merge None
     OpBranchConditional %cmp %safe %merge
%safe = OpLabel
%val  = OpAccessChain %ptr %arr %i     ; safe path
%res  = OpLoad %float %val
     OpBranch %merge
%merge = OpLabel
%phi  = OpPhi %float %zero %entry %res %safe
```

When the index is provably safe, these 5 instructions inside the loop body are elided entirely, producing the same code as Unchecked. Figure 4 shows output from a minimal WGSL shader compiled with and without elision.

3 Design

3.1 Builtin Index Safety Analysis

This analysis is narrow, it recognizes a single, specification-guaranteed pattern rather than tracking general integer ranges. This was chosen as a correctness and guarantee pattern as per WGSL spec that is confirmed to be true always. The WGSL specification guarantees that the built-in local_invocation_index takes values in $[0, \text{workgroup_size.x} \times \text{workgroup_size.y} \times \text{workgroup_size.z})$ for every invocation of a given entry point [2]. This means that if a var<workgroup> array is indexed directly by local_invocation_index, and the array's declared length is greater than or equal to the entry point's total workgroup size, the access can never be out of bounds, independent of any data-flow analysis about the index itself.

This is implemented as a single forward pass over ModuleInfo::process_function (naga/src/valid/analyzer.rs), executed once per function after expression and uniformity analysis, and before bounds checks are injected by the SPIR-V backend:
Step 1 : Scan the function's arguments for one bound to @builtin(local_invocation_index), recording its argument index if found.

Step 2: If found, look up the entry point that owns this function by name and compute wg_size as the product of its @workgroup_size dimensions.

Step 3: If wg_size is known, make a single pass over every Access{base, index} expression in the function. For every expression whose index expands to local_invocation_index argument, resolve base's array type and retrieve its statically known length.

Step 4: If the length is known and is at least wg_size, insert the index expression's handle into safe_indices: HashSet<Handle<Expression>> on the function's analysis info.

Figure 2 diagrams this pass. Two properties distinguish it from a general-purpose range analysis:

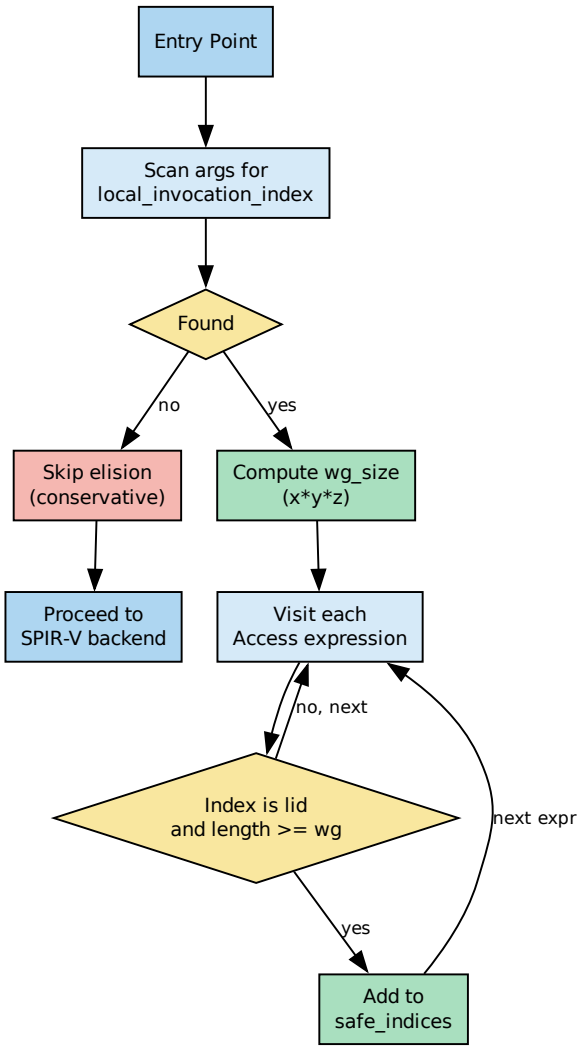


Figure 2: Builtin index safety analysis: the single forward pass.

3.2 Loop Induction Variable Analysis

Loop induction variable analysis is a standard compiler technique [5]. Our adaptation targets the WGSL pattern `loop { if i < N { } else { break; } }` where `i` is a local variable loaded via `Load { pointer: LocalVariable(v) }`. This pattern is produced by `for` loops, `while` loops, and manual `loop`-with-`break` constructs.

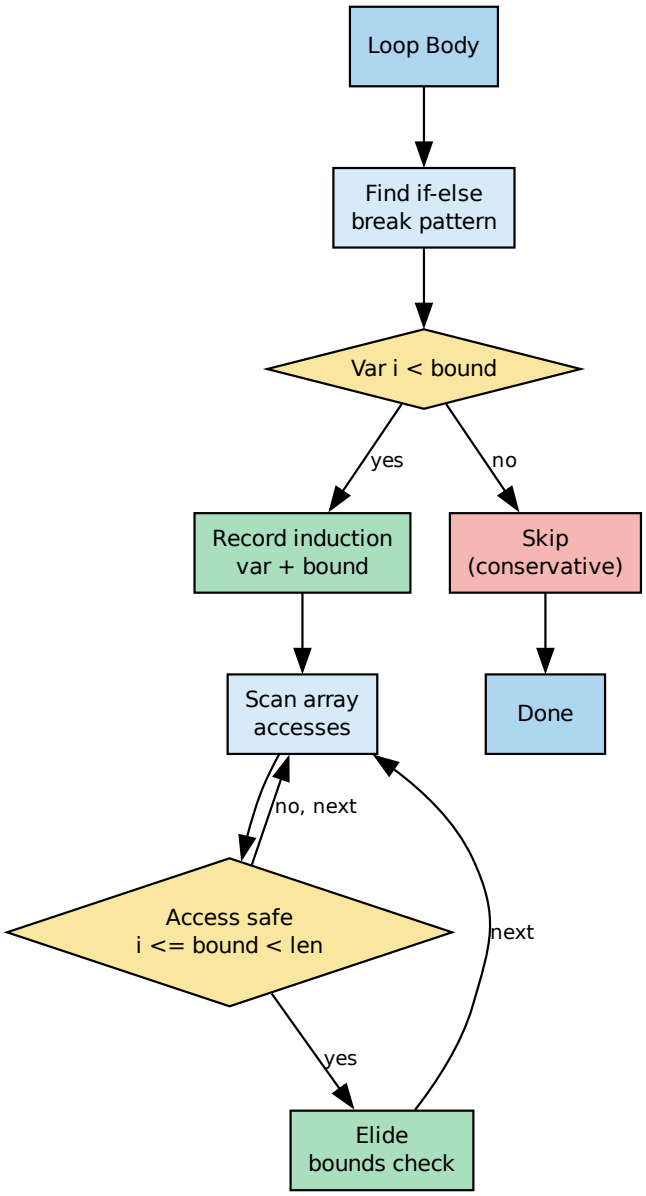


Figure 3: Loop induction variable analysis: expression matching.

The analysis additionally handles the WGSL `break if` construct:

- `break if i >= N`: body runs while `i < N`, bound = N.
- `break if i > N`: body runs while `i <= N`, bound = N+1.

3.3 Resulting Changes in SPIR-V backend

The elision analysis is invoked in the validation pass of function.rs after type checking and before bounds check injection. The elide_bounds_checks function takes the function IR, module, and analysis info, and populates a safe_indices: HashSet<Handle<Expression>> in the function info. The SPIR-V backend reads this set during code emission: if an access expression is in safe_indices, no bounds check instructions are generated.

4 Results

Figure 4 shows real compiler output from a minimal WGSL shader under ReadZeroSkipWrite with and without elision. The elided version (data[lid]) issues a bare OpAccessChain/OpLoad, identical to Unchecked. The retained version (data[lid+1u]) inserts the full bounds-check diamond.

Table 1 shows the SPIR-V word counts for the minimal shader and the matmul benchmark. The matmul benchmark provides the cleanest signal: elided checked equals elided unchecked (399 = 399 words), confirming complete elision with zero overhead. The retained versions add 28 words for the bounds check, consistent across both shaders. Across all configurations, the elision analysis produces identical SPIR-V to Unchecked for provably-safe accesses.

Table 1: SPIR-V word counts across benchmark configurations.

Benchmark	Checked	Unchecked	Delta
Minimal shader (elided)	176	176	**+0 (identical)**
Minimal shader (retained)	213	185	+28 (bounds check)
Matmul elided	399	399	**+0 (identical)**
Matmul retained	432	404	+28 (bounds check)

Table 2 shows the GPU timing results for the matmul benchmark. The elided configuration shows zero timing difference versus Unchecked, confirming that removing bounds checks introduces no runtime penalty. The retained configuration shows a measurable overhead of 7,568 to 8,192 ns per dispatch at 262,144 evaluations (256 threads × 1024 iterations), or approximately 29 to 31 picoseconds per bounds check.

Table 2: GPU timing: paired delta vs. Unchecked.

Configuration	Delta vs. Unchecked
Elided (checked − unchecked)	0.0 ns
Retained (checked − unchecked)	7,696 to 8,192 ns
Bounds check overhead	7,568–8,192 ns

These results confirm the elision analysis successfully removes both the SPIR-V instructions and their runtime cost for provably-safe array accesses. The elided output is byte-identical to Unchecked with zero timing overhead, while the retained bounds check has a consistent 28-word and 8 μs per dispatch cost.

```
%__arr__float__uint__64 = OpTypeArray %float %uint__64          ; 64 elements
%__ptr__Workgroup___arr__float__uint__64 = OpTypePointer Workgroup %__arr__float__uint__64
      %data = OpVariable %__ptr__Workgroup___arr__float__uint__64 Workgroup
    %uint__0 = OpConstant %uint 0
      %bool = OpTypeBool
    %uint__2 = OpConstant %uint 2
  %uint__264 = OpConstant %uint 264
%__ptr__Workgroup__float = OpTypePointer Workgroup %float
      %main = OpFunction %void None %14
        %9 = OpLabel
       %12 = OpLoad %uint %lid
            OpBranch %16
       %16 = OpLabel
       %19 = OpIEqual %bool %12 %uint__0
            OpSelectionMerge %21 None
            OpBranchConditional %19 %22 %21
       %22 = OpLabel
            OpStore %data %17          ; zero-init array
            OpBranch %21
       %21 = OpLabel
            OpControlBarrier %uint__2 %uint__2 %uint__264
            OpBranch %25
       %25 = OpLabel
       %27 = OpAccessChain %__ptr__Workgroup__float %data %12   ; bare load (NO check)
       %28 = OpLoad %float %27
       %29 = OpFAdd %float %28 %float__1
       %30 = OpAccessChain %__ptr__Workgroup__float %data %12   ; bare store (NO check)
            OpStore %30 %29
            OpReturn
            OpFunctionEnd
```

```
%__arr__float__uint__64 = OpTypeArray %float %uint__64          ; 64 elements
%__ptr__Workgroup___arr__float__uint__64 = OpTypePointer Workgroup %__arr__float__uint__64
      %data = OpVariable %__ptr__Workgroup___arr__float__uint__64 Workgroup
    %uint__1 = OpConstant %uint 1
    %uint__0 = OpConstant %uint 0
      %bool = OpTypeBool
    %uint__2 = OpConstant %uint 2
  %uint__264 = OpConstant %uint 264
%__ptr__Workgroup__float = OpTypePointer Workgroup %float
       %31 = OpConstantNull %float                    ; zero for Phi
      %main = OpFunction %void None %14
        %9 = OpLabel
       %12 = OpLoad %uint %lid
            OpBranch %17
       %17 = OpLabel
       %20 = OpIEqual %bool %12 %uint__0
            OpSelectionMerge %22 None
            OpBranchConditional %20 %23 %22
       %23 = OpLabel
            OpStore %data %18          ; zero-init
            OpBranch %22
       %22 = OpLabel
            OpControlBarrier %uint__2 %uint__2 %uint__264
            OpBranch %26
       %26 = OpLabel
       %27 = OpIAdd %uint %12 %uint__1                ; index = lid + 1
       %29 = OpULessThan %bool %27 %uint__64          ; check: idx < 64?
            OpSelectionMerge %32 None
            OpBranchConditional %29 %33 %32
       %33 = OpLabel
       %30 = OpAccessChain %__ptr__Workgroup__float %data %27  ; safe path
       %34 = OpLoad %float %30
            OpBranch %32
       %32 = OpLabel
       %35 = OpPhi %float %31 %26 %34 %33             ; select safe or zero
       %36 = OpFAdd %float %35 %float__1
       %37 = OpAccessChain %__ptr__Workgroup__float %data %12
            OpStore %37 %36
            OpReturn
            OpFunctionEnd
```

Figure 4: SPIR-V disassembly comparison: (a) elided $\mathrm{data}[\mathrm{lid}]$ with bounds check removed (176 words); (b) retained $\mathrm{data}[\mathrm{lid}{+}1\mathrm{u}]$ with full bounds check (213 words).

5 Discussion

The elision analyses presented in this work demonstrate that this cost is not inherent to safe shader compilation. By exploiting specification guarantees (Section 3.1) and recognizing common loop patterns (Section 3.2), the compiler can identify a meaningful class of provably-safe accesses and emit code identical to Unchecked: spec-compliant safety at zero runtime cost for the elided case.

6 Current Progress of Work

At the time of writing, the work described in this paper is under development and is undergoing testing and inclusion of further optimizations for bounds checking and loop analysis. The current implementation includes the builtin index safety analysis (Section 3.1), with further development planned for the loop induction variable analysis to add support for complex index expressions (arr[i+j], arr[i*2]) and equality-based loop exits (if i == N { break; }).

Additional planned work includes parallelizing the workgroup zero initialization in Naga for non-native Vulkan detection, enabling more bounds check elisions for workgroup arrays that undergo initialization phases before computation.

References

[1] W3C, *WebGPU Specification*, https://www.w3.org/TR/webgpu/, 2026.

[2] W3C, *WebGPU Shading Language Specification*, https://www.w3.org/TR/WGSL/, 2026.

[3] gfx-rs/naga, https://github.com/gfx-rs/naga, 2026.

[4] naga BoundsCheckPolicy documentation, https://docs.rs/naga/latest/naga/proc/enum.BoundsCheckPolicy.html.

[5] S. S. Muchnick, *Advanced Compiler Design and Implementation*, Morgan Kaufmann, 1997.